

CS/EE 147 — Assignment 3 Report

Kaushik Vada

May 2026

Q1 — Execution Time vs. Matrix Shape

Configuration	Kernel time (s)
1024×1024 (square)	0.004239
4096×256 (tall-skinny)	0.002788
256×4096 (short-wide)	0.002753

The square case is slowest because it has the largest output and performs the most total work. The rectangular cases produce smaller output matrices, so fewer thread blocks are launched and the kernel finishes faster.

Q2 — Execution Time vs. Tile Size

Tile size	Kernel time (s)
8×8	0.003168
16×16	0.003754
32×32	0.007077

Smaller tiles use less shared memory per block, allowing higher occupancy. The 32×32 tile requires 8 KB of shared memory per block, which limits the number of concurrent warps and hurts performance.

Q3 — Global Memory Access Patterns (64×64 , 16×16 Tiles)

With 64×64 matrices and tile size 16, there are $4 \times 4 = 16$ thread blocks, each iterating over 4 tile phases. In each phase, 256 threads each load one element of A and one of B from global memory. Total global reads: $16 \times 4 \times 256 \times 2 = 32,768$. Without tiling, each output element independently reads a full row and column, giving $64 \times 64 \times 128 = 524,288$ reads — about $16 \times$ more.

Q4 — Memory Bandwidth: Tiled vs. Non-Tiled

In a non-tiled kernel, every output element reads k values from A and k values from B directly from global memory. In the tiled version, each tile of data is loaded once into shared memory and reused by all T threads in that tile. This reduces global memory reads by a factor of T (the tile dimension), replacing slow DRAM accesses with fast on-chip shared memory reads.

Q5 — Total Arithmetic Operations for 64×64 Multiplication

Each of the $64 \times 64 = 4096$ output elements requires 64 multiply-add operations (2 FLOPs each):

$$\text{Total FLOPs} = 64 \times 64 \times 64 \times 2 = 524,288$$

Q6 — Shared Memory for 8×8 Double-Precision Tiles

Two tiles (one for A , one for B), each 8×8 doubles (8 bytes each):

$$2 \times 8 \times 8 \times 8 = 1,024 \text{ bytes} = 1 \text{ KB per block}$$

Q7 — Why Tiling Exploits Data Locality

Tiling stages data in shared memory so each global load is reused T times instead of once. This increases the arithmetic intensity from $O(1)$ to $O(T)$, shifting the kernel from memory-bound to compute-bound. Shared memory has roughly $100\times$ lower latency than global memory, so the reuse directly translates to a large speedup.

Q8 — Tile Size, Occupancy, and Register Pressure

Larger tiles require more shared memory and more accumulator registers per thread (one per tile row held in registers). Both reduce the number of warps that can run concurrently on a streaming multiprocessor, lowering occupancy. Smaller tiles improve occupancy but increase the ratio of synchronization overhead to useful computation. The optimal tile size balances these two effects for the specific GPU.